

1. Define a `BankAccount` class with fields `accountNumber` (String), `holderName` (String), and `balance` (double). Add a method `displayInfo()` that prints all details. In `main`, create two objects with different values and call `displayInfo()` on each.
2. Create a `Student` class with fields `name`, `id`, and `gpa`. Create three student objects and print which student has the highest GPA without using arrays.
3. A payment system has multiple payment methods (Credit Card, PayPal). Define an abstract class `Payment` with an abstract method `processPayment(double amount)`. Implement it in `CreditCard` and `PayPal` classes. In `main`, process a payment using each method.
4. Create an encapsulated `Employee` class with private fields `name`, `salary`, and `department`. Add getters for all fields and a setter for `salary` that rejects any value below 3000. In `main`, try setting a valid and an invalid salary.
5. A university has `Person` as a base class with `name` and `age`. `Student` extends `Person` and adds `studentId` and `major`. `GradStudent` extends `Student` and adds `thesisTitle`. Create a `GradStudent` object and print all its details (multilevel inheritance).

6. A `'Shipping'` class calculates shipping cost. Overload the `'calculateCost'` method: (1) takes weight only, (2) takes weight and distance, (3) takes weight, distance, and boolean for express shipping.
7. A notification system has a base class `'Notification'` with a `'send(String message)'` method. Override it in `'EmailNotification'`, `'SMSNotification'`, and `'PushNotification'`. Store all three in a `'Notification[]'` array and send a message to all of them in a loop.
8. Create a `'Product'` class with all 4 constructor types: (1) default sets `name="Unknown"`, `price=0`, (2) parameterized takes name and price, (3) copy constructor copies another product, (4) private constructor shown via a static factory method. Demonstrate each in `'main'`.